MDPI

Article

Open or Frontier? A Cost-, Latency-, and Energy-Aware Benchmark of Large Language Models for Software Vulnerability Detection

Firstname Lastname [1,*]

[1] Affiliation; e-mail@e-mail.com
* Correspondence: e-mail@e-mail.com

Abstract

(1) Background: Large language models (LLMs) are increasingly applied to software vulnerability detection, yet evaluations overwhelmingly report accuracy while ignoring the monetary and environmental cost of inference, and they under-represent openly available (open-weight) models relative to proprietary frontier systems. (2) Methods: We benchmark eight contemporary LLMs—three frontier (Claude Sonnet 5, GPT-5.1, Gemini 3.1 Pro) and five open-weight (DeepSeek-V3.2, GLM-5, Llama-3.3-70B, Qwen3-Coder-30B, Gemma-3-4B)—on a stratified 1549-function subset of the label-clean PrimeVul dataset, treating cost-per-task, latency, and energy as first-class evaluation axes alongside detection quality. Monetary cost and latency are measured directly; energy is estimated with a transparent FLOP-based model for the API-served models and measured directly on a controlled GPU for the two locally servable open models. (3) Results: On this imbalanced benchmark no model beats a trivial always-safe baseline on raw accuracy, so we evaluate with balanced accuracy. The three highest-quality models are all open-weight (balanced accuracy: DeepSeek-V3.2 0.674, Gemma-3-4B 0.649, GLM-5 0.647; vs 0.61–0.62 for the frontier models); on a Holm-corrected paired balanced-accuracy bootstrap, DeepSeek-V3.2 significantly exceeds all three frontier models, while the next two open models rank above the frontier with smaller, mostly not-individually-significant gaps. A 4-billion-parameter open model matched or exceeded frontier quality at roughly one-hundredth the monetary cost and one-to-two orders of magnitude less energy per task, and the open-weight models occupy the quality–efficiency Pareto frontier; absolute quality is modest for all models, at realistic prevalence every model yields low precision, and a classical static analyzer (Flawfinder) is competitive—underscoring the task's difficulty. Direct on-GPU measurement of two open models shows the FLOP estimate agrees to within roughly a factor of two and leaves the Pareto ordering intact. (4) Conclusions: For this task, detection quality and efficiency point the same way—toward small open-weight models—with direct implications for on-premises security tooling and its carbon footprint.

Keywords: large language models; vulnerability detection; software security; benchmarking; energy efficiency; open-weight models; inference cost; green AI

Received:
Accepted:
Published:

1. Introduction

Large language models (LLMs) have moved quickly from code completion into security analysis, and software vulnerability detection is a prominent target: the promise is an assistant that reads a function and flags whether it contains an exploitable weakness. A

growing body of work evaluates LLMs on this task [1–3], and dedicated benchmarks now exist across both defensive detection and offensive capability assessment [4]. Two blind spots persist, however, and together they motivate this study.

First, evaluations are almost exclusively accuracy-centric. In deployment, an organization that runs vulnerability triage over a large codebase cares not only whether a model is correct but what each verdict costs in money and energy, and how long it takes. The general LLM community has begun to treat efficiency as a first-class evaluation axis [11,14,15], but this machinery has not been imported into security-task benchmarking, where the environmental and economic footprint of inference remains essentially unreported [4].

Second, security evaluations lean heavily on proprietary frontier models. A recent review of offensive-security LLM studies found that proprietary systems appeared in nearly every publication while open-weight models were used in roughly half [4]. Yet open-weight models are precisely what many security teams need: on-premises deployment for privacy, compliance, and data-residency reasons, without sending sensitive source code to an external service [5].

This paper brings the two concerns together. We benchmark eight contemporary LLMs—a spread of open-weight and frontier systems—on realistic vulnerability detection, and we report cost, latency, and energy alongside accuracy. Our contributions are:

- A cost-, latency-, and energy-aware benchmark of open-weight versus frontier LLMs on the label-clean PrimeVul dataset [1], with a reproducible, test-driven measurement harness (Section 3).
- Empirical evidence that, on this task, small open-weight models sit on the quality–efficiency Pareto frontier: the best of them match or beat frontier models on balanced accuracy—with statistically significant gaps—at one-to-two orders of magnitude lower cost and energy (Section 4).
- A precise account of what can and cannot be measured for closed API models—direct measurement of cost and latency throughout, direct on-GPU energy measurement for the two locally servable open models, and a bounded, assumption-explicit FLOP estimate of energy for the API-only models that the measured points corroborate to within roughly a factor of two (Section 3).

We are explicit that these results are a first, deployment-realistic snapshot: energy is measured directly for the two locally servable open models and FLOP-estimated for the API-only models, and models are evaluated in a direct-answer configuration. Section 5 details these limitations and the extensions they invite.

2. Related Work

2.1. LLMs for Vulnerability Detection

Benchmarks for LLM-based vulnerability detection have proliferated. Meta's CyberSecEval suite evaluates insecure-code generation and broader cyber-safety [3]; SecVulEval provides a large, de-duplicated, CVE-grounded C/C++ set with statement-level labels [2]. Independent evaluations temper early optimism: Ullah et al. find that leading LLMs cannot yet reliably identify or reason about vulnerabilities [16], and Steenhoek et al. report function-level accuracy close to random [17]—a picture our results corroborate. A recurring and central concern is label quality: many widely used datasets carry substantial label noise and cross-split duplication, which inflates reported performance and collapses on realistic data [8,18]. PrimeVul was constructed specifically to address this, using near-human-accuracy labelers, rigorous de-duplication, and chronological splits, and it reports that strong code models drop from high F1 on noisy datasets to near-random on PrimeVul [1]. We adopt PrimeVul as our backbone for exactly this reason.

2.2. Efficiency and Energy of LLM Inference

Measuring the cost of inference is an active area. Direct energy measurement on datacenter GPUs has been reported for open models [9,10], and standardized power-measurement methodology exists at the systems level [11]. A well-documented pitfall is that NVIDIA's sampled power interface substantially under-samples short kernels, which motivates counter-based measurement [12]. For closed API models, energy cannot be measured directly; infrastructure-aware estimation frameworks bound it from public performance data and hardware assumptions [13]. Efficiency has also entered general LLM benchmarks as a first-class axis [14,15]. Our methodology reuses this rigor but applies it, for the first time to our knowledge, within a security-detection benchmark.

2.3. The Gap

Very recent work has begun to compare open and frontier models on security tasks and to report some efficiency figures—for example latency and coarse cost on web vulnerability detection [6], or cost and latency for a specialized fine-tuned detector [7]. None, however, reports energy as a co-equal per-task axis across a broad off-the-shelf open-weight roster. That combination—open-weight breadth, a security detection task, and cost/latency/energy together—is the gap this paper addresses.

3. Materials and Methods

3.1. Task and Dataset

We use binary function-level vulnerability detection: given a C/C++ function, the model answers whether it contains a vulnerability. We draw from the PrimeVul test split [1] (24,788 functions after loading; 549 vulnerable, a realistic 1:44 class ratio). Because PrimeVul's test split contains only 549 vulnerable functions, a label-stratified sample balances at most to 1098; we evaluate on a stratified sample of **N=1549** (all 549 vulnerable functions plus 1000 safe functions), fixed by seed for reproducibility. Function inputs are truncated to a fixed character budget to bound cost. Each model is prompted to answer with a verdict first, followed by an optional explanation; a lenient parser extracts the verdict, and outputs that yield no parseable verdict are recorded as parse failures rather than silently counted as "safe."

3.2. Models

We evaluate eight models spanning parameter scale and provenance (Table 1). All are served through a unified OpenAI-compatible API gateway. Frontier models are proprietary; open-weight models are publicly available and, in principle, self-hostable. Reasoning-capable models are run in a direct-answer configuration where the provider permits it; Gemini 3.1 Pro exposes no non-reasoning mode and is therefore run with reasoning enabled.

Table 1. Evaluated models. Active-parameter counts for frontier models are bounded assumptions used only for the energy estimate.

Model	Tier	Est. active params
Claude Sonnet 5	frontier	∼100B (assumed)
GPT-5.1	frontier	∼100B (assumed)
Gemini 3.1 Pro	frontier	∼100B (assumed)
DeepSeek-V3.2	open	∼37B active
GLM-5	open	∼32B active
Llama-3.3-70B	open	70B
Qwen3-Coder-30B	open	3B active (MoE)
Gemma-3-4B	open	4B

3.3. Measurement Methodology

We treat detection quality, cost, latency, and energy as co-equal evaluation axes. Because the open-weight models here are also accessed through the API gateway, cost and latency are measured identically across all models, giving a like-for-like comparison. Energy is FLOP-estimated for the API-served models and, for the two open models we could serve locally on a single GPU (Qwen3-Coder-30B, Llama-3.3-70B), measured directly on that GPU.

Cost is computed exactly from logged input/output token counts and published per-token pricing. **Latency** is measured client-side (total wall-clock and output tokens per second); we report it as service latency and note that the reported figures were collected under concurrent load and therefore conflate model speed with queueing. **Energy** cannot be measured for API-served models, which run on unknown hardware with unknown batching [13]. We therefore report a transparent FLOP-based estimate, $E \approx 2\, N_{\text{active}}\, T\, \varepsilon$, where N_{active} is the active parameter count, T the total token count, and ε a system-level energy-per-FLOP constant calibrated so that a frontier-class query reproduces published per-query estimates [13]. For open-weight models the active parameter count is known, giving a grounded estimate; for frontier models it is a bounded assumption, and we treat the resulting energy as an estimate with an explicit uncertainty rather than a measurement. Every result row carries an `energy_source` tag so that measured and estimated energy are never conflated. For the two locally servable open models we *measure* energy directly on a dedicated NVIDIA H200 GPU, reading the NVIDIA Management Library (NVML) cumulative energy counter [10,12] across each request at concurrency 1; we report both a gross energy and an active energy that subtracts the measured idle draw. Qwen3-Coder-30B was served in `bfloat16` and Llama-3.3-70B in FP8 (the 70B model does not fit a single H200 in `bfloat16`). The remaining open models could not be served under our fixed local stack—Gemma-3-4B is incompatible with the pinned vLLM/CUDA build, and DeepSeek-V3.2 and GLM-5 exceed single-GPU capacity—so their energy remains FLOP-estimated.

3.4. Harness and Reproducibility

All experiments run through a purpose-built, test-driven harness (75 automated tests) with pluggable model backends and measurement meters. Each run records its resolved configuration, random seed, model versions, and a per-token price snapshot; results are written incrementally so that long runs are crash-safe and resumable. Generation is deterministic (temperature 0, fixed maximum output length). As a non-LLM reference point we additionally run Flawfinder 2.0.20, a lexical C/C++ static analyzer, over the identical functions, labelling a function vulnerable when it raises at least one hit at risk level ≥ 1. The harness, configurations, and the exact sampled function identifiers are released with this paper (see Data Availability).

3.5. Use of Generative AI

In accordance with venue policy we disclose that generative AI tools were used substantially in this work: to assist with the research design, to implement the measurement harness, to run the experiments, and to draft this manuscript. All AI-generated code is covered by the automated test suite, all quantitative results are produced by the released harness from logged data, and the authors have reviewed and take responsibility for the content. No AI system was used to fabricate data or citations.

4. Results

4.1. Choice of Primary Metric

Our sample is imbalanced (549 vulnerable, 1000 safe), so raw accuracy is dominated by a trivial classifier: predicting "safe" for every function yields 0.646 accuracy, which exceeds the accuracy of *every* model we tested. Raw accuracy is therefore not a meaningful primary metric here. We instead lead with **balanced accuracy** (the mean of true-positive and true-negative rates), for which both trivial classifiers score exactly 0.5, and we report the Matthews correlation coefficient (MCC) and F1 alongside it. Metrics are computed over parsed predictions; parse rates were near-perfect (0.985–1.000) once reasoning-capable models were configured to emit a verdict rather than only internal reasoning.

4.2. Detection Quality

Table 2 reports all eight models plus the two trivial baselines, sorted by balanced accuracy. Three findings hold. First, the three highest-quality models are all open-weight—DeepSeek-V3.2 (0.674, 95% CI $[0.653, 0.696]$), Gemma-3-4B (0.649 $[0.626, 0.673]$), and GLM-5 (0.647 $[0.629, 0.664]$)—above all three frontier models (0.613–0.623). Because balanced accuracy, not raw accuracy, is our primary metric, we test the pairwise gaps with a stratified paired bootstrap of the balanced-accuracy difference (20,000 resamples, resampling the 549 positive and 1000 negative task outcomes independently so the design is preserved) and control the family-wise error rate across the nine open-versus-frontier comparisons with a Holm correction. DeepSeek-V3.2 significantly exceeds all three frontier models (Δ balanced accuracy $+0.053$ to $+0.062$; Holm-adjusted $p < 0.001$). The next two open models rank above every frontier model on the point estimate, but by smaller margins that are individually significant before correction yet mostly do not survive it: after Holm adjustment only Gemma-3-4B's advantage over Claude-Sonnet-5 remains significant ($+0.037$; $p = 0.046$), while its gaps to GPT-5.1 ($+0.036$; $p = 0.07$) and Gemini-3.1-Pro ($+0.025$; $p = 0.12$), and all three of GLM-5's gaps ($+0.013$ to $+0.025$; $p \geq 0.07$), do not. The robust claim is therefore that the single best open model significantly beats the entire frontier tier and that the three best open models all rank above it—not that every top-open-vs-frontier gap is individually significant, and certainly not that every open model beats the frontier: Qwen3-Coder-30B is indistinguishable from chance (balanced accuracy 0.516; MCC 0.030, 95% CI $[-0.02, 0.08]$).

Second, absolute quality is modest for all models (balanced accuracy 0.52–0.67, MCC 0.03–0.34), and specificity varies widely, from 0.25 (Llama-3.3-70B) to 0.57 (Qwen3-Coder-30B). This matters for deployment. On PrimeVul's natural 1:44 prevalence—rather than our balanced sample—precision is low for every model: 2.4–3.7%, only marginally above the 2.2% base rate an always-flag classifier would achieve. The balanced-accuracy ranking thus identifies the *least miscalibrated* model, not a deployment-ready one; we return to this in Section 5. Third, the frontier models in particular show high recall but low specificity (Gemini-3.1-Pro and Claude-Sonnet-5 flag $\approx$70% of safe functions), which is why they trail on the balanced metric despite competitive F1.

Table 2. Detection quality and efficiency on PrimeVul (N=1549, direct-answer mode), sorted by balanced accuracy. Balanced-accuracy 95% CIs and Holm-corrected paired significance tests are given in Section 4.2; **Spec.** is specificity (true-negative rate). Cost is measured for all models; energy is measured on a dedicated H200 GPU for the two locally served open models (†, active energy; see Section 4.4) and FLOP-estimated otherwise. Flawfinder is a lexical static-analysis reference point (any hit at risk level ≥1; cost/energy negligible). Best per column in bold, except the energy column (mixed measured/estimated) and specificity.

Model	Tier	Bal. acc.	MCC	F1	Acc.	Recall	Spec.	USD/task	Energy (J)
DeepSeek-V3.2	open	**0.674**	**0.343**	**0.614**	0.627	0.838	0.511	0.00017	526
Gemma-3-4B	open	0.649	0.290	0.587	0.612	0.778	0.521	**0.00004**	64
GLM-5	open	0.637	0.309	0.596	0.548	0.941	0.333	0.00048	424
Gemini-3.1-Pro	frontier	0.623	0.297	0.591	0.525	**0.963**	0.283	0.00448	1966
GPT-5.1	frontier	0.614	0.225	0.559	0.567	0.772	0.455	0.00139	1332
Claude-Sonnet-5	frontier	0.613	0.264	0.579	0.519	0.934	0.291	0.00441	2489
Llama-3.3-70B	open	0.603	0.260	0.578	0.502	0.956	0.251	0.00008	738†
Qwen3-Coder-30B	open	0.516	0.030	0.410	0.532	0.459	0.572	0.00006	88†
Flawfinder (static analysis)	—	0.589	0.235	0.377	0.682	0.271	0.907	≈0	≈0
Always-safe (baseline)	—	0.500	0.000	0.000	0.646	0.000	1.000	—	—
Always-vulnerable (baseline)	—	0.500	0.000	0.523	0.354	1.000	0.000	—	—

4.3. Efficiency and the Pareto Frontier

The efficiency separation is order-of-magnitude and robust to the choice of quality metric. Gemma-3-4B exceeds Claude-Sonnet-5 on balanced accuracy (0.649 vs. 0.613) while costing roughly one-hundredth as much per task ($0.00004 vs. $0.0044) and consuming one-to-two orders of magnitude less energy: the estimated 64 J vs. 2489 J is a 39× gap at our assumed frontier active-parameter count, and spans roughly 10–150× across the plausible range of that assumption (Section 4.4). Service latency, measured client-side but under concurrent load (and so conflating model speed with queueing—see Section 5.3), points the same way: Gemma-3-4B is the fastest model at 1.4 s mean wall-clock per task, against 3.4–4.7 s for the slowest four (DeepSeek-V3.2, GLM-5, Gemini-3.1-Pro, Claude-Sonnet-5). Figures 1 and 2 plot balanced accuracy against cost and energy on log axes: the small open-weight models occupy the upper-left (high quality, low cost/energy), and no frontier model is Pareto-optimal on either axis.

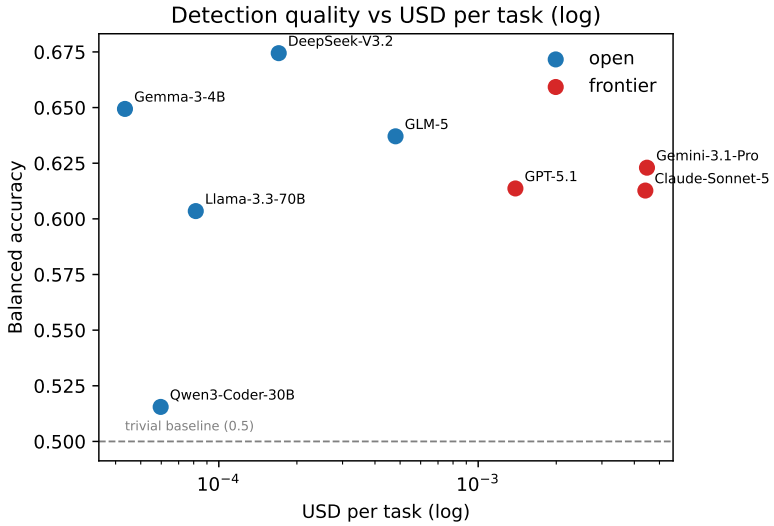

Figure 1. Balanced accuracy versus measured monetary cost per task (log scale). Open-weight models (blue) dominate the frontier models (red); the dashed line marks the trivial-baseline balanced accuracy of 0.5.

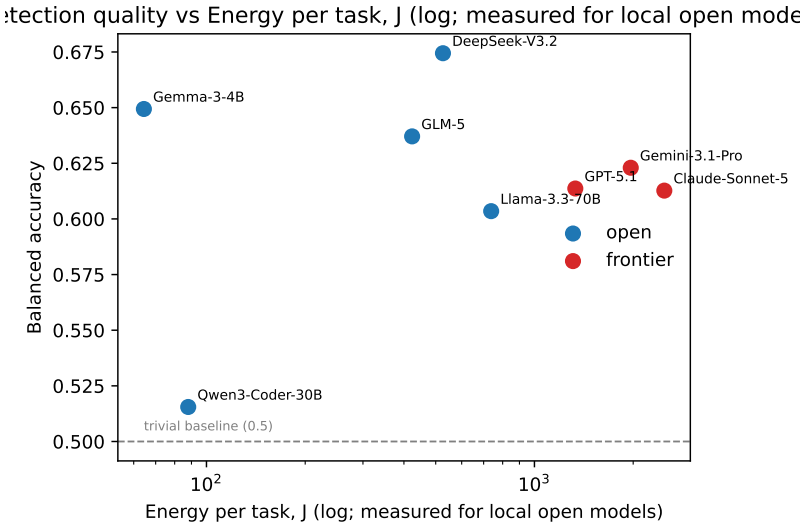

Figure 2. Balanced accuracy versus energy per task (log scale). Energy is measured on an H200 GPU for Qwen3-Coder-30B and Llama-3.3-70B (Section 4.4) and FLOP-estimated for the other six models; substituting the measured values for the prior estimates leaves the ordering unchanged.

4.4. Measured Versus Estimated Energy

For the two open models we could serve locally, we replace the FLOP estimate with direct H200 measurement and ask how far the estimate was off (Table 3).[1] The estimate agrees with the measurement to roughly a factor of two (0.8×–2.2×), and errs in opposite directions by architecture. For the sparse Qwen3-Coder-30B (3B active of 30B total) the estimate *undershoots* the measurement by 2.2× (40 vs. 88 J): an active-parameter FLOP count captures neither the memory traffic of the full expert set nor the low GPU utilization of single-request decode. For the dense Llama-3.3-70B, served in FP8, the estimate *overshoots*, at 0.8× the measurement (944 vs. 738 J), because the estimate is precision-agnostic while the measurement benefits from 8-bit arithmetic. Measured energy per output token—1.4 J for the 3B-active MoE and 11.6 J for the 70B model—*straddles* the ∼3–4 J/token reported for datacenter GPUs by Samsi et al. [10]: neither value lands inside that band, but they bracket it and scale with model size as expected, which is the most this loose cross-hardware check can establish.

Two cautions bound what this validates. It confirms the estimator's structure and its energy-per-FLOP constant for small and mid-size *open* models under single-request serving; it does not test the assumed frontier active-parameter count (Table 1), which is the dominant uncertainty in the frontier energy figures and cannot be measured here. And the measurement is at concurrency 1—a low-utilization, energy-pessimistic regime—while the estimator's constant is calibrated to batched datacenter serving, so exact agreement is not expected. The robust, measurement-independent conclusion is narrower and is the one we rely on: the measured values move both open models only within their order of magnitude, keeping them well below every frontier estimate and leaving the Pareto frontier and the open-versus-frontier separation unchanged (Figure 2); the only reordering is at the extreme low-energy end, where Gemma-3-4B (estimated 64 J) rather than Qwen3-Coder-30B (measured 88 J) is now the least energy-intensive model. The efficiency gap is so large that even a two-fold estimation error, in either direction, does not threaten it.

[1] The measurement is a dedicated $N = 300$ run at concurrency 1; the "Estimated" column is the mean FLOP estimate from the main $N = 1549$ run. Mean output length matched across the two (≈64 tokens), so the per-task comparison is not materially confounded by the differing subset, but the two are not the identical requests.

Table 3. Measured (H200, NVML) versus FLOP-estimated energy per task for the two locally served open models. Active energy subtracts the measured idle draw; the ratio is measured-active over estimated.

Model	Precision	Estimated (J)	Measured active (J)	Meas./est.	J / out. tok
Qwen3-Coder-30B	bfloat16	40	88	2.2×	1.4
Llama-3.3-70B	FP8	944	738	0.8×	11.6

5. Discussion

5.1. Open-Weight Models on the Efficiency Frontier

The central result is that, for this task, the quality-optimal and the efficiency-optimal choices coincide, and both are open-weight. On balanced-accuracy-per-dollar and per-Joule, small open models such as Gemma-3-4B dominate every frontier system by one to two orders of magnitude, and no frontier model is Pareto-optimal (Figures 1, 2); this conclusion is strongest on the *measured* cost axis, where it needs no energy assumption at all. It comes with an essential qualification: as Section 4.2 shows, every model is near chance in absolute terms, and at PrimeVul's realistic 1:44 prevalence even the best model raises roughly twenty-six false positives per true positive. The finding is therefore conditional—*if* an organization deploys an LLM for function-level triage, the small open models are both the higher-quality and the far cheaper choice—rather than an endorsement of the task as solved. We also flag that our cost figures are the API gateway's prices for every model; a genuine on-premises deployment substitutes amortized hardware, electricity, and utilization for that price, so the break-even volume above which self-hosting actually wins is a modeling question we leave to future work.

Translating energy to carbon makes the environmental stake concrete while keeping it bounded. At a representative grid intensity of 250 gCO_2eq/kWh and a datacenter PUE of 1.2, Gemma-3-4B's 64 J per task is about 5 $mgCO_2eq$, against roughly 210 $mgCO_2eq$ for Claude-Sonnet-5; over a million-function codebase that is ≈5 kg versus ≈210 kg (and ≈10 vs. ≈390 kg at the world-average 475 gCO_2eq/kWh). These are operational GPU-energy figures only—they exclude embodied hardware carbon, which at low on-premises utilization would count against the smaller-model case—so we treat them as illustrative rather than a life-cycle assessment. The absolute footprints are modest in isolation, but the *ratio* is large and recurring, compounding with every re-scan of an evolving codebase: a concrete instance of the "green AI" argument [9,14] applied to a security workload. The result is not that open models are uniformly superior—Qwen3-Coder-30B is indistinguishable from chance—but that the best open models dominate the frontier on both the quality and the efficiency axes.

5.2. Why the Benchmark Is Hard

No LLM exceeded 0.68 balanced accuracy, and none beat the trivial always-safe classifier on raw accuracy—an important sanity check on this imbalanced set. A classical lexical static analyzer, Flawfinder, offers an external reference point (Table 2): it reaches 0.589 balanced accuracy—below every LLM except the near-chance Qwen3-Coder-30B, and on par with the weaker frontier models—and, because it flags conservatively (specificity 0.91), it is the only method here that beats the always-safe baseline on raw accuracy (0.682 vs. 0.646) and the most precise at realistic prevalence (6% vs. the LLMs' 2.4–3.7%), at essentially zero cost and energy. That a decades-old pattern matcher lands in the same league as billion-parameter models underscores how unsolved the task remains. Rather than a defect, this difficulty reflects PrimeVul's construction: by removing the label noise and duplication that inflate performance on older datasets [1,8,18], it exposes how far

current LLMs remain from reliable function-level vulnerability detection. Several models default to flagging—Gemini-3.1-Pro and Claude-Sonnet-5 among the frontier systems, and GLM-5 and Llama-3.3-70B among the open ones, all with specificity below 0.36—which in a triage setting translates into alert fatigue rather than useful signal. DeepSeek-V3.2 and Gemma-3-4B lead the balanced metric because they pair competitive recall with specificity above 0.5; Qwen3-Coder-30B is also conservative (specificity 0.57) but couples it with recall barely above chance, so it gains nothing.

5.3. Limitations and Threats to Validity

Several limitations bound our claims and shape the measured extensions we are pursuing.

- **Energy is only partly measured.** We measure energy directly on an H200 GPU for the two locally servable open models; for the three frontier models (unknown hardware) and the three open models we could not serve locally, energy remains a FLOP-based estimate. The measured points agree with that estimate to within roughly a factor of two and preserve the Pareto frontier and open-versus-frontier separation (Section 4.4), but they validate only the open-model regime—the assumed frontier active-parameter count, which drives the frontier energy magnitudes, is untested, so we report those magnitudes as a sensitivity range rather than point values.
- **Direct-answer configuration.** Reasoning was disabled where the provider allowed it, to obtain a comparable single-shot judgment; Gemini 3.1 Pro admits no such mode. A dedicated reasoning-versus-direct comparison—quantifying how much accuracy reasoning buys and at what energy cost—is a natural and, for a cost-aware venue, valuable follow-up.
- **Decoding-budget asymmetry.** The two reasoning-only frontier models (Claude-Sonnet-5, Gemini-3.1-Pro) need a larger output budget (256 vs. 64 tokens) to emit a verdict and are reported at that budget; as they are already the efficiency losers, their reported cost/energy gap is conservative. GLM-5, which we initially ran at the larger budget, is re-run at the matched 64-token budget with reasoning disabled and reported there: its balanced accuracy is essentially unchanged (0.637 vs. 0.647), confirming the budget does not drive its result.
- **Latency under concurrency.** The service-latency figures we report (Section 4) were collected under concurrent load and conflate model speed with queueing; a controlled single-request latency pass is future work, so we treat latency as a directional observation rather than a precise axis.
- **Training-data contamination.** PrimeVul is built from public CVE-fixing commits—all here from 2008–2022, predating the models' training cutoffs—that may appear in pretraining data. Binning the vulnerable functions by CVE year reveals a modest memorization gradient: recall on pre-2020 CVEs is 7–15 points higher than on 2021–2022 CVEs for the better-balanced models (e.g. GPT-5.1 0.88 vs. 0.73; DeepSeek-V3.2 0.90 vs. 0.81), consistent with some memorization of older, well-documented vulnerabilities. The gradient appears in both tiers, so it does not obviously explain the open-versus-frontier gap, but it does suggest absolute scores may be optimistic; a cleaner bound would restrict to post-cutoff CVEs or to PrimeVul's paired vulnerable/patched variants (which resist superficial memorization), and is planned.
- **Single sample, single run.** Results use one stratified draw of the 1000 safe functions, one seed, and one generation per item at temperature 0. The paired bootstrap (Section 4.2) quantifies within-sample uncertainty, but because the balanced-accuracy ranking is specificity-driven, resampling the safe pool and repeating generations

would additionally bound draw-to-draw and API run-to-run variance for the closely-spaced middle of the table.

- **Baselines.** We include a lexical static-analysis baseline (Flawfinder) alongside the trivial classifiers, but not a fine-tuned learned detector (e.g. LineVul [19]) or a semantic analyzer such as CodeQL; adding these would further calibrate the absolute difficulty and is a natural extension.
- **Scope.** We report one dataset, one task formulation, and a balanced sample of 1549 functions; generalization to other languages, to statement-level localization, and to the realistic-imbalance regime (via a scored metric such as VD-Score [1]) remains future work.

None of these caveats reverses the headline: the direction of both quality and efficiency favors small open-weight models; cost is measured for all models, energy is measured for two of them and agrees with the estimate to within roughly a factor of two, and the efficiency gaps are one-to-two orders of magnitude—far larger than any of these uncertainties.

6. Conclusions

We benchmarked eight open-weight and frontier LLMs on label-clean vulnerability detection, measuring cost, latency, and energy alongside detection quality. Open-weight models led on quality and dominated on efficiency: the best open model significantly exceeds every frontier model on balanced accuracy (DeepSeek-V3.2 beats all three; the three best open models all rank above the frontier), at roughly one-hundredth the cost and one-to-two orders of magnitude less energy per task, while all models remained close to chance on this hard benchmark. That last caveat is load-bearing: at realistic prevalence every model raises far more false alarms than true ones, so the practical claim is conditional—*if* an LLM is deployed for function-level triage, the small open models are the accuracy- and efficiency-motivated choice. Direct H200 measurement of two locally servable open models shows the FLOP estimate agrees to within roughly a factor of two and leaves the Pareto ordering intact; extending on-GPU measurement to the remaining self-hostable models, adding non-LLM baselines, and quantifying the accuracy–energy trade-off of reasoning are the immediate next steps.

Author Contributions: Conceptualization, methodology, software, investigation, and writing were carried out by the author(s) with generative-AI assistance as disclosed. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The measurement harness, all run configurations, the exact sampled function identifiers, and the aggregated results are released at [repository URL]. The PrimeVul dataset is available from its original authors [1].

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Ding, Y.; et al. Vulnerability Detection with Code Language Models: How Far Are We? In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025; arXiv:2403.18624.
2. Ahmed, M.B.U.; et al. SecVulEval: Benchmarking LLMs for Real-World C/C++ Vulnerability Detection. *arXiv* 2025, arXiv:2505.19828.
3. Bhatt, M.; et al. Purple Llama CyberSecEval: A Secure Coding Benchmark for Language Models. *arXiv* 2023, arXiv:2312.04724.
4. Happe, A.; et al. Benchmarking Practices in LLM-driven Offensive Security: Testbeds, Metrics, and Experiment Design. *arXiv* 2025, arXiv:2504.10112.

5. Levi, M.; et al. Toward Cybersecurity-Expert Small Language Models. *arXiv* 2025, arXiv:2510.14113.
6. Neef, S.; et al. Evaluating LLMs for Real-World Web Vulnerability Detection. *arXiv* 2026, arXiv:2606.21397.
7. Dahiya, V.; et al. Are Frontier LLMs Ready for Cybersecurity? Evidence for Vertical Foundation Models from Dual-Mode Vulnerability Benchmarks. *arXiv* 2026, arXiv:2605.23243.
8. Croft, R.; Babar, M.A.; Kholoosi, M.M. Data Quality for Software Vulnerability Datasets. In *Proceedings of ICSE*, 2023; arXiv:2301.05456.
9. Luccioni, A.S.; Jernite, Y.; Strubell, E. Power Hungry Processing: Watts Driving the Cost of AI Deployment? In *Proceedings of ACM FAccT*, 2024; arXiv:2311.16863.
10. Samsi, S.; et al. From Words to Watts: Benchmarking the Energy Costs of Large Language Model Inference. In *Proceedings of IEEE HPEC*, 2023; arXiv:2310.03003.
11. Tschand, A.; et al. MLPerf Power: Benchmarking the Energy Efficiency of Machine Learning Systems. *arXiv* 2024, arXiv:2410.12032.
12. Part-time Power Measurements: nvidia-smi's Lack of Attention. *arXiv* 2023, arXiv:2312.02741.
13. Jegham, N.; et al. How Hungry is AI? Benchmarking Energy, Water, and Carbon Footprint of LLM Inference. *arXiv* 2025, arXiv:2505.09598.
14. Liang, P.; et al. Holistic Evaluation of Language Models (HELM). *arXiv* 2022, arXiv:2211.09110.
15. Niu, C.; et al. TokenPowerBench: Benchmarking the Power Consumption of LLM Inference. *arXiv* 2025, arXiv:2512.03024.
16. Ullah, S.; et al. LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2024; arXiv:2312.12575.
17. Steenhoek, B.; et al. A Comprehensive Study of the Capabilities of Large Language Models for Vulnerability Detection. *arXiv* 2024, arXiv:2403.17218.
18. Chakraborty, S.; Krishna, R.; Ding, Y.; Ray, B. Deep Learning based Vulnerability Detection: Are We There Yet? *IEEE Transactions on Software Engineering* 2022; arXiv:2009.07235.
19. Fu, M.; Tantithamthavorn, C. LineVul: A Transformer-based Line-Level Vulnerability Prediction. In *Proceedings of the IEEE/ACM 19th International Conference on Mining Software Repositories (MSR)*, 2022.